

MInDes 软件开发入门手册

王梓航* 黄奇†

2024 年 9 月 26 日

目录

1	简介	4
2	Windows平台开发	4
2.1	开发环境搭建: Microsoft Visual Studio的安装	4
2.1.1	Microsoft Visual Studio的项目属性设置	5
2.1.2	MInDes在Microsoft Visual Studio项目中的文件添加与 初次运行	7
2.2	开发环境搭建: Microsoft Visual Studio Code	9
2.3	结果可视化: Paraview	10
3	Linux平台开发	10
3.1	WSL的安装	10
3.2	Linux的基本操作	11
3.2.1	文件管理	11
3.2.2	软件安装	13
3.2.3	编辑器操作	14
3.3	配置Linux C++开发环境	15

*主笔, E-mail: w.zihang@qq.com

†审阅, E-mail: qihuang0908@163.com

3.4	Linux C++环境下的程序调试	16
3.4.1	环境测试	16
3.4.2	Linux C++环境下的MInDes初次运行	17
4	Git & GitHub 协作	18
4.1	分布式版本控制工具: Git	18
4.1.1	Git的核心思想: 记录变更	18
4.1.2	Git基础概念	19
4.2	GitHub Desktop操作	20
4.3	GitHub: 代码托管平台	21
4.4	MInDes的Git(GitHub)仓库	21
5	概念介绍	22
5.1	编辑器 & 编译器 & IDE	22
5.2	文档(Documentation), API Reference	23
5.3	编程语言, Python & C++	23
5.4	文件后缀	24
5.5	文件编码	26
5.6	控制字符	27
5.7	面向对象	27
5.8	C++语言标准	28
5.9	编译	28
5.10	编译器, MSVC, G++ 与 C++标准委员会	28
5.11	GNU Compiler Collection, GCC & G++	29
5.12	代码构筑系统, CMake	30
5.13	代码调试(Debug)	30
5.14	并行运算, OpenMP与MPI	31
5.15	Linux 内核与Linux 系统	32
5.16	操作系统, WSL & 虚拟机	32
5.17	命令行, 终端, Shell, Powershell & Bash	33

6	Q&A	33
6.1	我是搞材料的,学这么多计算机的软件的内容做什么?	33
6.2	你写的一大堆全都是泛泛而谈,有没有更专业的文档?	34
6.3	你写的这些长到看不下去,有没有省流版?	36
6.4	感觉看完还是云里雾里的...	37
6.5	这么多页,一幅图都没有?	38
6.6	我觉得你写得不好,我要自己写/修改/增补内容.	38
6.7	我看了这个手册,也搞定了里面讲的配置,但是还是不知道怎 么开始做MInDes开发,这个不是开发入门手册吗?	38

1 简介

MInDes (Material Intelligent Design) 程序的开发以运行高效、结构清晰、功能完善为目标, 因涉及第三方库以及多平台开发等原因, 对软件的开发环境有一定的要求. 本文将介绍 Windows平台的开发环境搭建, Linux平台开发环境搭建, Windows与Linux平台的程序调试, 基于Git与GitHub的多人协同等, 并在文末补充介绍一些程序开发的基本概念以及一些可能常见的问题. 旨在分享必要的程序开发知识, 帮助读者快速上手入门程序开发. 欢迎对本文进行补充, 遇到的疑问也欢迎同笔者交流沟通.

笔者的部分软件并未进行汉化, 涉及具体操作时将贴上英文以及自行翻译的中文以提供对照. 翻译的中文可能有所差错, 敬请谅解. 文中出现的序号0均意为检查项目, 请检查该项以确保接下来的步骤的顺利完成. 文中使用不同的字体以区分不同的对象, 以下是对照条目.

- 斜体(楷体) *italic* 指代文件, 运行库. 同时也作为强调之用.
- 黑体 **boldface** 指代文件夹.
- 无衬线体 **sans serif** 指代程序中出现的图标文字等. 通常有笔者翻译的中文和英文对照. 其中出现的箭头 > 指代目录的层级结构.
- 灰底衬线体 **code** 指代命令行中输入的命令或是按键快捷键.

截至本手册成稿时, 笔者并没有新设备来一一检验文中所述均为无错漏或多余的必要步骤, 如有何处错漏, 敬请批评指正. 如需要快速搭建好开发测试环境, 请移步至章节6: Q&A 部分查阅您需要的内容.

2 Windows平台开发

本节介绍Windows开发需要的基本环境搭建.

2.1 开发环境搭建: Microsoft Visual Studio的安装

Windows平台推荐使用Microsoft Visual Studio (以下称VS) 的最新社区版 (VS Community 2022) 进行开发. 下面将给出VS的安装步骤:

0. VS是一个功能齐全, 体积庞大的IDE, VS本体的下载与安装时间较长, 请耐心等待. 在安装VS前请预留好充足的空间. 安装过程中可能需要您重启计算机.
1. 点击 [此处](#) 可打开 VS 官网, 选择下方的下载即可下载VS的安装器. 请注意, 这一步下载的仅为VS的安装器, 并非VS的本体.
2. 在安装好VS的安装器后, 双击打开便会先进入一段时间的检查, 请耐心等待. 随后安装器便会打开安装界面.
3. 在开发组件中, 请选择**桌面应用和移动应用(Desktop & Mobile)** > 使用**C++桌面开发(Desktop Development with C++)**. 然后请点击右下方的安装.
4. 这之后会有一段时间的组件下载和安装的过程, 耗时较长, 请耐心等待.
5. 在安装完成后, VS 会要求尽快重启计算机. 请自行重新启动计算机以完成安装.

通过上面的步骤, 您将在您的计算机上安装好Microsoft Visual Studio.

2.1.1 Microsoft Visual Studio的项目属性设置

随后我们演示如何为一个VS项目做属性设置.

0. 请确保正确安装了VS, 特别是VS的组件, 应包含C++桌面开发的组件.
1. 打开VS, VS会先弹出一个外观配置页面, 请根据自己的喜好选择. 随后VS会弹出一个欢迎窗口, 左侧为历史项目(目前是空的), 右侧有若干选项. 请选择第四项**创建新项目(Create a new project)**.
2. VS内置了多种项目模板. 但我们需要一个空项目模板, 因此在右侧选择**空项目(Empty Project)**, 并点击下一步.
3. 接下来VS会要求设置新项目的名称, 路径位置等. 请根据自己喜好填写, 并点击右下方**创建(Create)**.

4. 随后便会进入VS的主界面。第一次进入可能会弹出一个花里胡哨的‘What’s New’, 可以直接关掉。现在创建一个新的C++文件。VS中的多种文件统称为项(item)。在**解决方案浏览器(Solution Explorer)** (可能在页面右方) 中选择**源文件(source)**文件夹(过滤器)后右键, 在**添加(Add)**子菜单中选择**新项(New Item...)**, 然后再在弹出的对话框中为该文件命名。现在该文件便会打开在主页面中, 等待编辑。
5. 编辑该文件使之成为一个可运行的C++代码。下面给出一个简单的C++源文件示例:

```
#include <iostream>
int main() {
    std::cout << "hello, world! from Visual Studio";
}
```

您可以将上述代码复制粘贴。编辑完成后保存。

6. 现在修改项目的属性, 首先修改C++的语言标准。右键单击**解决方案浏览器(Solution Explorer)**中与您项目名称相同的一项, 点选属性(Properties), 此时将弹出项目属性对话框。
7. 在该对话框左上方的配置中选择**所有配置(All Configuration)**, 在左侧栏中找到**配置属性 > 通常(Configuration Properties > General)**, 右侧界面中选择“C++ 语言标准”, 更改为**ISO C++ 17 标准(/std:c++17) (ISO C++ 17 Standard(/std:c++17))**, 然后在右下角点击**应用(Apply)**。
8. 再在左侧栏中找到**配置属性 > C/C++ > 语言(Configuration Properties > C/C++ > Language)**选项, 在右侧的界面中点击**Open MP 支持(Open MP Support)**一项的右侧, 选择**是(/openmp) (Yes(/openmp))**, 为该项目添加OpenMP的支持, 然后在右下角点击**应用(Apply)**。最后点击右下方的**确定(OK)**以保存设置并退出该页面。
9. 此时, 您可以点击上方的**本地Windows调试器(Local Windows Debugger)**。随后控制台(黑框)将会跳出并显示您的程序输出。该程序应为在

屏幕上打印“hello, world! from Visual Studio”. 这便表示您的环境已经配置完成且成功运行.

至此, 您已为在VS中为开发MInDes所需要的项目属性设置. 请注意, 这些属性设置是仅针对单个项目的, 如有别的新的项目需要类似设置, 请按照上述类似步骤进行设置.

2.1.2 MInDes在Microsoft Visual Studio项目中的文件添加与初次运行

假设您现在通过GitHub等手段获得了MInDes的源代码, 现在希望创建一个新的VS项目或将之添加到一个已有的VS项目中. 以下是将MInDes源文件添加至VS以及初次运行 MInDes的具体步骤.

0. 请确保VS已经被正常安装, 且您已知晓如何设置项目属性.
1. 请打开MInDes源码的文件夹放在一边, 并打开或创建一个空的项目.
2. 在VS中, 将解决方案浏览器中的所有过滤器(带有小漏斗的文件夹图标)删除, 并右键项目名称, 选择**添加 > 新过滤器(Add > New Filter)**, 此时会创建一个新的过滤器, 请将其名称命名为MInDes源码文件夹中的**solvers**, 或**modules**.
3. 以上述步骤的类似方式逐步建立多级过滤器, 名称按照源文件的文件夹中所设置的一般.
4. 在建立好所有过滤器后, 将对应文件夹的内容拖入对应的过滤器上, 即可完成对项目添加源文件.
5. 请在源文件夹中找到**lib**文件夹, 将之复制到项目所在文件夹中, 然后找到 **lib/x64** 下的三个文件: *libfftw3-3.dll*, *libfftw3f-3.dll*, *libfftw3l-3.dll*, 将之复制到项目所在的文件夹中. 这一步是为了能让程序编译时能找到这静态链接库以及三个运行时库(动态链接库). 举例而言, 假设您的项目文件夹是 **C:\Project**, 您的GitHub库中的源文件夹位于 **D:\GitHub\MInDes_DEV** 中, 则您需要将 **D:\GitHub\MInDes_DEV\lib** 文件夹复制到 **C:\Project** 中, 然后再从 **C:\Project\lib\x64** 中找到文

件`libfftw3-3.dll`, `libfftw3f-3.dll`, `libfftw3l-3.dll`, 复制到文件夹 **C:\Project** 中.

6. 在完成上述所有步骤后, 您就已经完成了MInDes源文件的添加. 尝试运行该程序. 运行前请勿忘记项目属性的设置, 以免出现不必要的报错.
7. 点击VS上方工具栏中的绿色实心箭头, 该按钮将启动代码的调试/运行. 由于程序体量较大, 程序的编译过程可能需要一段时间等待. 开始编译后, 您会在VS的文本编辑器下方看到新的窗口, 此时应该会显示编译过程的信息. 如果程序编译无误, 随后会跳出一个控制台框, 其即为正在运行的该程序, 而VS的界面也会发生变化以供用户调试程序. 若程序编译有误, 则刚刚显示编译过程信息的窗口将显示出现的错误. 请就报错的说明文字和错误代码进行搜索以排查错误.

至此, MInDes在Windows端VS上的开发环境配置便完成了. 此处再列出若干第一次运行时可能出现的问题(描述使用英文, 请对照对应的错误代码):

- *LNK1104 cannot open file 'lib/x64/libfftw3-3.lib'* 表明您没有将静态库正确放置在您的项目文件夹下, 导致链接器找不到静态库而报错. 请检查上述第5步.
- 程序启动, 但弹出窗口 **The code execution cannot proceed because libfftw3-3.dll was not found. Reinstalling the program may fix this problem.** 这表明程序在运行时未能找到动态链接库, 导致缺少关键组件而报错. 请重新检查第5步, 将对应动态链接库挪至正确位置.
- *C1128 number of sections exceeded object file format liimit: compile with /bigobj* 表明在调试过程中生成的对象文件(object file) 的大小不满足条件, 需要使用 `/bigobj` 的 flag 进行编译. 请打开项目设置页面, 找到**配置属性** > **C/C++** > **命令行属性页面** (Configuration Properties > C/C++ > Command Line)中的**附加选项**(Additional Options), 在其中输入“`/bigobj`”后点击右下方**应用**(Apply)保存配置.
- 报了许多错误, 有若干个警告. 请检查警告中是否有 *STL4038 The contents of <filesystem> are available only with C++17 or later.* 这条. 如果

有, 请参考上文环境配置部分, 正确设置该项目采用的C++语言标准. 请注意, 您可能是仅为某个配置(如, 仅为Debug)设置了这些项目. 您可以考虑将设置应用在所有配置中.

2.2 开发环境搭建: Microsoft Visual Studio Code

除了使用Visual Studio进行Windows端开发外, 建议使用Microsoft旗下的Visual Studio Code (以下简称VSCode)进行文件浏览, 特别是程序输入文件的编写. 后续在Linux平台的开发也需要依赖VSCode的远程连接和WSL功能. 点击 [此处](#) 可打开 VSCode 官网.

VSCode在安装时, 请注意在 **Select Additional Tasks** 中选择 **Add “Open with Code” action ...** 的两个选项, 以便在安装后可以通过右键打开文件夹和文件. 如果您想要为所有用户安装(比如, 安装到 *C:\Program Files\Microsoft VS Code*), 请在下载界面中选择 *System Installer* (第二个选项)并使用管理员权限安装.

VSCode以其丰富的插件生态而闻名, 安装插件可以在侧边栏选择或 **Ctrl+Shift+X** 打开插件市场, 在页面上方框中输入相应关键词即可检索相关插件. 这里推荐若干插件以提高工作效率:

- **Chinese (Simplified)(简体中文) Language Pack for Visual Studio Code:** 可以使VSCode的语言显示变为中文显示.
- **WSL:** 添加链接WSL的支持以便于从VSCode中打开WSL并进行程序调试
- **MInDes-LSP:** 笔者开发的为MInDes输入文件提供语法支持的VSCode插件. 可以从 [该Github仓库](#) 中下载安装. 安装方法为在插件市场界面右上侧的三个点处展开菜单, 选择**从VSIX安装(Install from VSIX...)**, 在弹出的对话框中选择下载好的VSIX文件即可.
- **C/C++ Extension Pack:** 为C或C++语言提供语言支持, 包括语法高亮, 语法纠错, 断点调试等功能, 其中还包含 **CMake** 插件.
- **CMake:** 提供在VSCode中使用CMakeLists.txt进行程序构筑的支持.

其中前三个推荐在Windows平台的VSCode中安装, 如果没有在Windows端使用VSCode调试程序的需求, 其余的可以选择只在WSL中安装即可.

2.3 结果可视化: Paraview

在使用MInDes进行模拟后, 会得到对应的模拟结果. MInDes选择使用 .vts格式存储运行结束后的相场演化图像, 该文件可以通过 Paraview这一开源数据可视化软件打开. 点击 [此处](#) 下载Paraview.

3 Linux平台开发

本节介绍Linux平台开发, 包括基础概念, 需要的环境搭建, 搭建环境的测试等.

3.1 WSL的安装

首先介绍WSL的安装. 通过WSL, 即可从Windows系统中简单地获得Linux环境进行开发测试. 详细步骤推荐查阅 [微软官方文档](#), 此处对该文档进行简要汇总.

0. WSL需要安装在Win10或Win11系统中

1. 使用Powershell或命令提示符 (cmd), 输入 `wsl --install`, 即可安装默认的Ubuntu发行版. 此过程中可能需要您提供若干次管理员权限. 随后您需要重启计算机以完成WSLUbuntu的安装.
2. 重启后会自动弹出Ubuntu的窗口(重启后若未启动, 则请于命令行中输入 `wsl` 或 `ubuntu`), 此时会提示输入用户名和密码, 此处用户名应仅使用小写字母, 数字或短横杠(减号), 且使用小写字母开头. 输入密码时密码不会显示于屏幕上, 这是正常的.
3. 上述账户将成为管理员账户, 在使用 `sudo` (Super User Do)命令时会需要输入前述设置的密码.

4. 使用命令: `sudo apt update && sudo apt upgrade` 对软件包管理器进行更新和升级, 以免无法正常安装和更新软件.

在上述步骤后, 一个可用的Linux发行版应已成功安装到WSL中. 使用时可以通过从Windows开始菜单中启动WSL或Ubuntu(默认安装), 也可从命令行启动(输入 `wsl`, 默认安装时也可输入 `ubuntu`).

3.2 Linux的基本操作

本小节将大致介绍在使用WSL时, Linux下的常用基本操作, 包括文件管理, 软件安装, 编辑器等.

3.2.1 文件管理

由于WSL其特殊的模式, 用户文件是与windows共享的, 意即大部分的文件管理操作在WSL中操作结果和windows中也会生效, 反之亦然. 但为了操作的便捷性, 此处介绍最常用的概念以及命令以便日常使用.

首先先介绍常用概念:

- 路径 (Path): 指文件、文件夹或程序等在计算机中的存储位置. Linux中的路径与Windows有所不同, 主要区别在于路径分隔符: Linux使用斜杠 / , 而Windows使用反斜杠 \ , 且文件存储位置也有细微区别.
- 工作文件夹 (Working Directory) 指当前用户所处的文件夹位置, 也可称为当前路径. 通过输入命令 `pwd` 即可查看当前路径.
- 相对路径 (Relative Path): 指从当前路径开始寻址的路径. Linux中使用 `.` 指代当前路径, 使用 `..` 指代上一级文件夹, 由此便可使用相对路径访问文件系统中任意一处文件地址.
- 绝对路径 (Absolute Path): 与相对路径对应, 指计算机中每个文件的唯一地址.
- 根目录 (Root Path): 指Linux文件系统最底层一级目录, 所有绝对路径从根目录开始. 根目录用 `/` 指代.

- 挂载文件夹 (mnt): 该文件夹位于根目录 `/` 下. 在Linux中所有的磁盘均被抽象为文件夹存储于mnt中. 因此, WSL中若要访问Windows系统中已有文件, 如路径为 `D:\Work\MInDes\test.txt` 的文件, 其在WSL中的路径则为 `/mnt/d/work/mindes/test.txt`. 其中, 第一个 `/` 即指根目录. 可以注意到linux系统文件路径不区分大小写.
- 用户文件夹 (User Directory): 指用户相关文件的存储路径, 类似于 Windows 中的 `C:\User` 文件夹. 在Linux中, 常使用 `~` 缩写用户文件夹. 一般进入系统时会自动进入用户文件夹中.

其次是常用命令, 通常以英文缩写的形式出现:

- `clear` (CLEAR screen): 用于清除屏幕输出. 建议克制其使用以保留屏幕输出历史, 一般在使用该命令后会难以寻回原先的输出历史.
- `cd` (Change Directory): 用于更改当前工作文件夹, 相对路径或绝对路径均可接受为参数.
- `ls` (LiSt): 用于列出文件夹中的内容. 无参数时列出当前文件夹内容(相当于 `ls .`), 也可接受相对路径或绝对路径作为要列出内容的目标文件夹. 且可接受 `-l` (以列表形式列出文件), `-a` (列出所有文件(包括隐藏文件))等参数.
- `cp` (CoPy): 用于将目标文件/文件夹中内容复制到另一个地址. 例如, `cp /mnt/d/work/test1 /mnt/d/test/` 会将文件/文件夹 `/mnt/d/work/test1` 复制一份到 `/mnt/d/test/` 文件夹中.
- `mv` (MoVe): 用于将目标文件/文件夹中内容移动到另一地址. 用法类似于 `cp`. 值得注意的是, `mv` 命令在检测到被移动对象没有改变位置时, 会根据第二个命令中的文件/文件夹名称来重命名目标文件/文件夹. 例如, `mv ./test1 ./mvtest` 会将工作目录下的 `test1` 文件重命名为 `mvtest`.
- `mkdir` (MaKe DIRectory): 用于创建文件夹, 如仅有名字则创建在当前工作文件夹下, 也可接受相对路径或绝对路径描述的文件夹名称.

- **touch**: 用于创建新文件或改变文件的时间戳. 如参数给出文件已存在, 则会改变它的最后一次修改的时间, 而如该文件不存在则会创建一个以该名称命名的文件.
- **cat**: 用于读取文件内容并输出到命令行中. 可以使用 **cat** 来查看文件内容而不会打开或修改文件本身.
- **rm** (ReMove): 用于删除文件/文件夹. 该命令极其危险, *Linux*系统中删除文件会直接从硬盘中抹除所有数据, 几乎是不可能恢复的, 因此请在任何时候都谨慎使用该命令. 在删除空文件夹时需要使用参数 **-d**, 如文件夹非空则需要使用参数 **-r** 来递归地删除所有文件夹下所有内容. 受系统保护的文件将需要使用 **sudo** 命令来以系统管理员身份执行, 而参数 **-f** 则会无视任何 **rm** 命令在执行过程中的报错, 意即强制执行. 在Linux中, ***** 通常用作通配符, 可以匹配任何字段. 因此, **请永远不要执行命令 `sudo rm -rf /*`**, 该命令会以管理员身份删除根目录下的所有内容, 也就是抹除掉所有磁盘内的任何数据. 除非你很清楚自己在做些什么.

3.2.2 软件安装

此处介绍一款软件包管理器: **APT**(Advanced Package Tool), 其常见于Debian系操作系统, 如Ubuntu, Debian等. 其余包管理器如pacman, yum等不在此处介绍.

启动APT可以在命令行中输入 **apt-get**, 也可缩写为 **apt**. 由于安装软件属于系统层面操作, 在涉及使用 **apt** 修改系统, 如安装, 卸载软件/软件包时, 需要使用管理员权限启动 **apt** (在 **apt** 前添加 **sudo**). 在初次使用 **apt** 时, 应首先更新软件源 (**sudo apt update**) 以及更新 **apt** 本身 (**sudo apt upgrade**).

为安装某个软件包, 此处以 **G++** (Linux下的常用 C++ 编译器) 为例, 演示如何安装软件包.

0. 确保 **apt** 的软件源与软件本体都得到了更新. 否则很容易因 **apt** 记录的软件源太旧, 找不到可用的软件源而安装失败.

1. 输入 `sudo apt install g++`, 系统会要求输入管理员密码. 该密码为安装WSL时所设定的用户与对应的密码. 输入密码时密码不显示在屏幕上正常现象.
2. 正确输入密码后系统将弹出软件大小等信息, 并询问是否安装. 输入 `Y`, `y` 或直接回车以进行安装.
3. 等待安装完毕后, 输入 `g++ --version` 以检查是否安装成功. 如若安装成功则会显示版本信息, 若未安装成功则会显示类似于 `-bash: g++: command not found` 的信息.

以上便是使用 `apt` 命令安装软件的流程. `apt` 可安装的除了一般意义上的软件外, 还包括运行库等. 值得注意的是, 安装库时名称可能与常用名称不同. 如需要查看已安装的软件/库, 可以输入 `apt list --installed`. 这会列出所有已安装的内容. 如要在此基础上检索, 如检索含有字段 `make` 的内容, 可以输入命令: `apt list --installed "*make*"`, 其中 `*` 指代任意字符.

3.2.3 编辑器操作

出于可能的使用需求, 此处简要介绍Linux自带的编辑器Vim, 即便开发过程中多数时间推荐使用的为微软出品的VSCode.

Vim是一款成熟的, 开源免费的, 基于命令行界面的文字编辑器. 可能与大多数编辑器所不同的是, Vim存在“模式”, 如编辑模式, 命令模式, 选择模式等. 在打开Vim时(命令行输入 `vim`), 会进入Vim的命令模式, 此模式下不可直接输入文字, 键盘键入会被处理为命令, 如 `j` 为向下翻页, `k` 为向上翻页, `o` 为向下插入新的一行并进入编辑模式, `i` 为在光标前进入编辑模式以插入文字等. 编辑模式的使用与通常文本编辑器类似, 而要回到命令模式则按下键盘 `Esc` 键即可. 要退出Vim则需要在命令模式中输入 `:q` (冒号和q).

该编辑器由于对鼠标依赖性小, 可自定义快捷键丰富等原因被一些程序员喜爱, 且由于历史原因常作为Linux系统的常用编辑器, 但其操作较为复杂, 学习门槛较高. 在拥有合适环境的条件下可以使用微软出品的VSCode以进行操作, 后者拥有更加现代化的操作逻辑, 更符合大多数使用者的习惯. 要打

开VSCode, 可以从Windows中打开VSCode后连接到WSL(下文将介绍), 或在环境配置结束后(下文将介绍)在命令行中输入 `code` 以启动VSCode.

3.3 配置Linux C++开发环境

为实现Linux系统下的C++开发环境配置, 需要安装以下的软件/程序包: G++, GDB, CMake. 同时为开发MInDes, 还需要安装运行库: *fftw*, *readline*. 其中软件的安装直接输入其名称即可, 安装运行库时名称有所不同, 在安装*readline*时请将名称替换为*libreadline-dev*, 其余操作相同. 而*fftw*的操作则更加复杂, 涉及自行编译等过程. MInDes程序的GitHub库中已有该运行库, 可以不自行安装. 如您对FFTW感兴趣, 希望自行安装在本地, 请参考FFTW组织的介绍: [FFTW Packages for Linux](#)

如若采用VSCode + G++ + GDB + CMake的工具链模式进行Linux平台下的程序调试, 请按照如下步骤进行配置:

0. 请确保在Windows下安装了VSCode与WSL, 且在Linux下正确安装了G++, GDB, CMake等软件工具. 为确认软件在Linux下已安装, 可输入 `g++ --version`.
1. 在Windows内打开VSCode, 按下快捷键 `Ctrl+Shift+X` 或打开插件(Extensions)侧边栏, 搜索“WSL”并安装. 请注意这里安装的是VSCode的插件, 和Windows中安装的WSL不同, 二者需要协同工作.
2. WSL插件安装完成后, VSCode界面最左下角会出现交错相对的箭头图标, 鼠标悬停其上时显示**打开远程窗口(Open a Remote Window)**. 点击此处即可在屏幕上方打开WSL对话框, 在对话框中选择**连接到WSL(Connect to WSL)**. 也可按下 `Ctrl+Shift+P` 后在屏幕上方对话框中输入**连接到WSL(Connect to WSL)**. 在等待些许时间后即可连接到WSL, 此时VSCode窗口上方与左下角将显示连接到WSL的发行版名称.
3. 此步将为WSL上的VSCode安装若干插件. 同第1步中的操作, 安装插件C++ Extension Pack.

至此便可完成Linux下的环境配置.

3.4 Linux C++环境下的程序调试

3.4.1 环境测试

为测试您的Linux C++环境已经搭建完毕可待使用, 请按照如下步骤:

0. 首先请确认自己已经安装好了前述软件.
1. 通过命令行在您希望的位置创建一个文件夹, 如在用户文件夹下创建一个名为**test**的文件夹: `mkdir ~/test`, 并进入该文件夹(以该例子, `cd ~/test`).
2. 用VSCode打开该文件夹: `code .`, 其中 `.` 指代了当前文件夹. VS-Code可能会询问是否要信任该文件夹, 请选择信任, 否则插件可能无法正常工作.
3. 按下快捷键 `Ctrl+Shift+P` 后输入并选择 **CMake: Quick Start**, 跟随提示首先输入工程名称(可填**test**), 随后选择 **C++**, **Executable**, 后续选项可以 `Esc` 跳过.
4. 此时VSCode将自动配置一个基础文件, 名为 `main.cpp`, 以及一个对应的CMake配置文件 `CMakeLists.txt`, 且VSCode的CMake插件将调用CMake 程序编译该文件, 结果将输出在名为**build**的文件夹中.
5. 在**main.cpp**文件中, 在内容为 `std::cout << "Hello, from test!\n";` 的一行的行号(可能是第4行, 可能会有变化)左侧点击, 将会出现一个红点(断点).
6. 随后点击VSCode界面最下方一行的小虫子图标进入调试环节, 此时界面下方将跳出调试台, 右上角出现有着若干按钮的控制程序运行的小框, 程序将停在断点前. 切换下方调试台到旁边的终端, 然后在右上角点击暂停的小箭头(或按下键盘快捷键 `F5`, 请注意不是 `F` 和 `5`)让程序继续运行.
7. 程序将运行结束, 在终端处显示输出的结果: `Hello, from test!` 以及一些别的信息.

若以上步骤均未出现问题, 说明Linux下以VSCode为宿主的G++ + GDB + CMake工具链已配置成功.

3.4.2 Linux C++环境下的MInDes初次运行

在您搭建好Linux C++开发环境且测试无误后, 假设您已经通过GitHub等手段获得了MInDes的源代码和相关文件, 便可尝试运行MInDes了. 下面将给出初次试运行的步骤:

0. 请确认所有相关内容放在一个单独的文件夹中, 或是源代码获得自Git. 请确保内容的完整性.
1. 请您通过VSCode打开这些文件所在的文件夹(本地Git仓库, 后文会提及), 例如, `code \path\to\your\repository\`. 请注意, 是在Linux端用VSCode打开, 而非在Windows端下打开.
2. 进入VSCode界面后, 在左侧文件浏览器界面中找到`CMakeLists.txt`并打开. 该文件记录了构筑MInDes所需要向编译器给出的指示.
3. 打开`CMakeLists.txt`后VSCode界面下方会出现VSCode自带的输出(OUTPUT), 调试台(DEBUG CONSOLE), 终端(TERMINAL)等的窗口, 并在输出窗口中弹出若干信息. 该信息由CMake提供, 用以提示CMake的工作状态.
4. 在输出窗口不再跳出信息后说明CMake已经配置完毕, 现在请点击VSCode界面最下方的小虫子图标尝试进行编译与调试. 点击后会开始编译源代码, 请耐心等待.
5. 如若编译正确, 没有报错, 您将在看到下方的界面自动跳转到调试台. 这里您可以与GDB交互. 调试台的右侧即为终端, 运行中的程序会出现在此处. 与VS不同, VSCode默认不会将程序运行在独立的控制台窗口中, 而是运行在默认集成控制台中. 您可以根据自己的喜好在VSCode中设置, 具体请查阅VSCode文档.

6. 如果编译失败, 请检查输出窗口左侧的**错误(PROBLEMS)**窗口, 其中会汇总编译过程中发生的警告(Warnings)与错误(Errors). 请查阅相关资料或网络搜索以解决报错.

经过以上步骤且若编译正确无误, 您便已经完成了MInDes在Linux端的初次运行.

4 Git & GitHub 协作

本节介绍Git与GitHub, 以及如何通过二者进行协同工作.

4.1 分布式版本控制工具: Git

Git是强大的分布式版本控制工具, 由Linus Torvalds与其主导的Linux开发小组在开发Linux内核时为实现方便的协同工作与版本控制而开发. 有一篇优秀的博客文章记录了相关的历史故事, 感兴趣可以看看: [Git的故事](#).

分布式版本控制工具的名称里, 分布式指可以去中心化地将仓库存储于多个位置, 而版本控制则指通过技术手段, 记录仓库的更改变动等, 且在必要时回滚版本到旧版本的软件工具. Git近乎是目前世界上最流行的版本控制工具, 其功能强大, 操作简单易用, 受到许多开发者的欢迎. 本节将尝试对Git做出简单的介绍, 包括其核心思想, 基础概念和基本操作等. 有一篇出色的教程较为详细地介绍了Git, 可以作为参考: [Git教程](#). 不过该教程主要介绍的是命令行操作, 相比之下易用性不如一些GUI程序. 本节将介绍 GitHub Desktop这一软件以便进行相关操作. 值得注意的是, GitHub Desktop并不是使用Git的唯一方式, 许多软件都集成了Git的功能, 例如VS, VSCode, CLion等, 且通过命令行操作Git也不失为一种好的手段.

4.1.1 Git的核心思想: 记录变更

Git从来不复制文件, Git只负责记录仓库内发生了什么变化. 通过为每一次提交所产生的变更做出记录, 赋以一个唯一的标识码, 使用者可以轻易追溯到所有的改动历史. 并且通过“分支”(Branch)功能, 用户可以从某次提交记录开始新创立一个分支, 在该分支做出的变动只会记录在该分支上, 不会影

影响到其余分支. 当用户希望将一个分支发生的内容变更记录同步到另一个分支时,便可使用“合并”(Merge)功能. 总之, Git只记录由Git管理的仓库内发生过什么变化, 不会去记录文件本身. 这就是Git的核心思想, 也是Git在不涉及网络环境时工作效率极高的原因.

4.1.2 Git基础概念

除了上一小节所提到的Branch, Merge以外, Git中有一些专有的, 约定俗成的概念, 这里对其一并介绍:

- repository(仓库): 指由Git接管的文件夹. Git无法管理一个普通的文件夹, 需要将普通文件夹用Git初始化为仓库后才可以被Git所管理.
- stash: 类似于中转站的存储位置. 仓库内的所有的改动并不会立刻被记录, 而是会先存储在stash中. 修改stash中的内容较为简单, 通过此方法可以让Git只记录用户认为准备妥当的改动, 避免提交内容混乱.
- commit(提交): 在确认stash内的改动无误后, 即可提交这次的变更记录. 请注意每次的提交Git都强制要求用户为提交进行命名, 不进行命名会导致提交失败. 但也请不要因此而随意起名, 具有描述性的提交命名可以帮助在回滚版本时快速大致了解每次变更, 从而快速定位问题所在.
- branch(分支): 由Git控制的变更记录串, 在一个branch上做出的改动不会影响到其余的branch. 这一特性给予Git强大的协作能力, 不同的用户在共同工作时可以创建一个自己的分支, 这样自己的工作不会受他人影响, 也不会影响他人.
- merge(合并): 该操作会使Git尝试将一个branch上产生的变更合并到另一个branch上. 如果两个branch的变更记录没有冲突的话便可将前一branch的变更同步至后一branch. 当冲突发生时, Git将会将决定权交由用户决定保留哪些变更.
- push(推送): 可以将本地Git记录的仓库变更推送到远程的库中, 常见的如推送至GitHub(后文将介绍). 如此便可使Git兼顾云端协作功能.

- **clone(复制)**: 与**push**相对, 可以将远程仓库的内容复制下载至本地.
- **fetch**: 可以将云端内容同步至本地. 请注意**fetch**与**clone**之间的区别, **clone**对一个仓库而言一般只操作一次, 而后这个库将通过**fetch**获取云端的内容以同步.
- **pull request**: 利用Git进行协作的基础即为使用**pull request**功能. 开发者通过克隆他人的仓库到本地, 经过修改变更后, 可以选择提交到自己的库中, 也可以选择将自己的变更提交给被克隆的仓库以供该仓库的原管理员审阅. 这一操作即为**pull request**. 这一过程类似于代码审核过程, 由原仓库管理员选择性合并他人提交的**pull request**以保证仓库的更新质量.

4.2 GitHub Desktop操作

GitHub Desktop是由GitHub组织开发的, 可以方便地与GitHub进行连接并实现**push**, **clone**, **fetch**与**pull request**等操作, 同时其也兼具Git的基本功能, 适合轻度使用者用来完成基础的版本控制与多人协作.

首先请下载并安装GitHub Desktop, 点击 [此处](#) 即可打开GitHub Desktop的官网. 安装后打开软件, 下列步骤将引导在本地创建一个Git仓库:

0. 请确保软件安装完毕.
1. 打开软件后于左上角**文件(File)**菜单中选择**新建仓库(New Repository)**或按下快捷键 **Ctrl+N**. 此时弹出对话框.
2. 在弹出的对话框中, 为仓库起一个名字, 可选地填入描述性信息, 然后选择仓库的位置. 请注意, 这里的仓库位置是仓库文件夹所在的位置, 而不是仓库的路径. 比如仓库名为**test**, 路径为 **D:\Repositories**, 则创建的仓库文件夹的绝对路径为 **D:\Repositories\test**.
3. 创建仓库完成后, 在仓库文件夹中做出一些改动, 如新建文件, 改变文件内容之类. 随后便可在GitHub Desktop中看到做出的改变.

4. 在GitHub Desktop左下角的框中写入提交信息, 包括提交名, 提交简述等, 随后点击**提交(Commit)**, 即可完成一次变更提交. 此时在左上角标签页**历史(History)**中即可看到改动的记录.

以上便可创建一个Git接管的仓库并创建一次变动提交. 关于创建分支等功能此处不再赘述.

4.3 GitHub: 代码托管平台

即使Git的分布式特性强大且突出, 这并不妨碍开发者们希望有一个值得信赖的可以放置自己代码的地方. 经过多年的发展, GitHub成为了这个众多开发者都选择的代码托管平台. 由于GitHub的域名“被墙”, 国内网络并不能直接访问GitHub的服务器. 此处仅介绍GitHub与Git如何协作.

Git的clone功能可以将仓库从一个远程地址下载到本地. 通过GitHub Desktop即可以实现该操作. 这个远程仓库可以是任意的提供相关服务的服务器, 常见的就是GitHub 的服务器. GitHub Desktop可以选择将自己的云端的库clone至本地, 也可以选择通过输入URL(网页地址)的方式, 将自己或他人仓库的URL输入后下载到本地. 选择他人仓库时, GitHub Desktop可能会询问这个仓库是留作自用还是用以提交pull request, 请根据需求选择, 这会影响GitHub Desktop在push时是更新自己账户下的仓库还是向原仓库提交 pull request.

GitHub本身也有诸多功能, 且在GitHub中的开源项目众多, 是学习编程乃至其他方面的优秀选择. 此处不再赘述.

4.4 MInDes的Git(GitHub)仓库

此处介绍MInDes软件代码的大致构成, 以便检查您拥有的代码是否完整. 如果您使用Git从GitHub库内拉取, 则应该会直接获取到完整的内容.

以下是MInDes_DEV的**main**分支中的内容.

- **infile**: 内含可供MInDes读取使用的测试输入文件与示例输入文件.
- **lib**: 包含所有的在程序运行或编译过程中需要使用到的静态或动态链接库.

- **modules:** 包含程序中各个模块的源代码.
- **solvers:** 包含程序中求解器主体的源代码.
- *AUTHORS* : 记录有软件作者信息.
- *COPYRIGHT* : 记录了程序的版权页
- *COPYRIGHT_HEADER* : 记录了附在源代码前的版权信息
- *MInDes_EXE.cpp* : 程序的主入口源文件.
- *README.md* : 程序的自述文件.
- *compile_exec.sh* : 使用G++以直接编译程序的Shell脚本.

以下是MInDes_DEV的**develop**分支的额外内容.

- *CMakeLists.txt* : CMake的配置文件, 用以CMake在Linux端构筑.
- *MInDes_DEV Change Log.md* : 手动维护的一份变更记录文件
- *cmake_build.sh* : 使用CMake直接编译程序的Shell脚本.

5 概念介绍

此处对一些概念做简要介绍, 可以作为选读内容.

5.1 编辑器 & 编译器 & IDE

编辑器指文本编辑器, 常见的如微软的记事本工具、Notepad++, linux下的Vim、Emacs, 多端通用的VSCode等;

编译器指将C、C++等程序代码编译为二进制可运行文件的程序, 常见的如随Visual Studio安装而安装的MSVC, linux下常用的GNU Compiler Collection, llvm组织出品的Clang等. 另一个常见的与之更类似的概念为解释器, 其为Python一类解释型语言所使用, 特点是从脚本逐行解释代码内容, 而不似编译器, 读取整个文件后编译为一个二进制可执行程序.;

IDE 为 Integrated Development Environment的缩写, 即集成开发环境, 可以看作是将编辑器和编译器结合起来的产物. 著名的Visual Studio即为集成开发环境, 此外还有JetBrains 公司出品的 CLion.

目前, MInDes使用Visual Studio开发Windows端, 使用VSCode + G++ + GDB + CMake的组合开发Linux端程序.

5.2 文档(Documentation), API Reference

程序开发领域的文档, 一般指帮助用户使用软件的用户手册, 亦或者记录软件功能的记录用手册. 在使用软件的过程中, 遇到的大多数问题都可以通过查询文档解决. 文档的书写风格通常不如教程来的简单易懂, 但文档会力求记录内容的准确性. 因此, 文档的编写某种程度也是程序开发不可缺少的一环, 而拥有较好的查阅文档的能力也有助于使用软件或开发程序.

API Reference(参考)是与文档类似的概念, 所谓API指Application Programming Interface, 因此API Reference旨在向读者介绍程序开发中每一个函数/方法的调用方式, 以助读者使用需要的API完成编程工作. 与文档相比, 更倾向于给开发者使用的手册.

5.3 编程语言, Python & C++

如今已经有众多的编程语言可供选择. 不同的编程语言有着不同的使用方法, 语言特性, 语法特点等. 目前常见的编程语言有Python, C, C++, Java, C#等等. 此处选择介绍Python和C++.

Python是一种面向对象的高级程序语言, 其语法灵活简单, 且因为丰富的三方库生态而充满活力. Python程序常由编写好的Python脚本(后缀常为.py)开始, 经由python解释器进行逐行运行后给出程序结果, 因此也易于修改程序中的错误(Debug), 且现已有很多成熟的交互式Python编程方式, 如Jupyter等, 便于进行科学研究中的数据处理等. 但由于其内部函数封装层次太高, 运行效率通常不如更贴近机器层的C, C++等.

C++曾经作为C语言的面向对象版本出现, 而经过长期的开发已与C语言有了极大的区别. 然而C++仍然保留了许多C语言的特性, 和C语言在许多方面依旧是兼容的, C++语法相比起Python较为复杂, 且调试过程也较为复杂,

但其运行效率更高, 更适合大规模的数据运算, 因而MInDes选择使用C++进行开发.

5.4 文件后缀

计算机上存储着许许多多的文件, 有些文件是单纯的文本文件(其实也并不单纯, 见下节), 有些文件则是二级制编码, 有些“文件”则是可以执行的应用程序. Windows系统为了区分这些不同的文件, 采用了在文件名最后添加一个以点号分隔开来的额外名称, 称为所谓的“后缀名”. 后缀名的存在能帮助用户以及操作系统辨认某个文件是哪一类文件, 以及应该采用什么样的软件打开或编辑该文件. 下面列出了部分Windows系统下常见的文件后缀和文件类型, 以及可能的打开方式.

- *.txt*: 常见的文本文件, 可以使用各种文本编辑器软件打开, 如记事本(Notepad), VSCode, Notepad++等.
- *.exe*: 常见的可执行二进制文件, 在Windows系统上一般可以直接双击打开.
- *.pdf*: Adobe公司开发的便携式文档格式(Portable Document Format)文件. 浏览器即可打开该类文件, 也有专用的PDF编辑器或PDF阅读器.
- *.doc/.docx*: 一类常见的富文本文件格式, 可以使用Word或WPS等支持富文本处理的软件打开. 有无*x*代表该文档的内部格式, 有*x*代表使用了更广泛采用, 软件间兼容性更好的XML格式. Word对两种格式是兼容的.
- *.cpp/.cxx/.cc*: 常见的C++源文件格式. 使用各种文本编辑器即可打开.
- *.h/.hpp/.hh*: 常见的C/C++头文件格式. 使用各种文本编辑器即可打开.
- *.dll*: 动态链接库(Dynamic Link Library). 是不可执行的二进制文件, 一般也无法打开或编辑.
- *.lib*: 静态链接库static link LIBrary. 同上, 是不可执行的二进制文件, 一般也无法打开或编辑.

- `.o`: 编译过程中产生的对象文件Object. 也是不可执行的二进制文件, 一般也无法打开或编辑.

后缀名种类繁多, 这里不再举例. 需要提醒的是, Windows下文件的名称中是允许包含点号的, 这部分的点号不会影响文件的后缀, 而是作为文件自身的名字存在. 这种情况下请注意Windows所显示的文件类型. Windows通常会读取文件的后缀, 然后将文件类型显示在文件的属性栏中. 如果希望避免后缀名而引起的乌龙, 请在Windows 文件浏览器中进行相关设置以使其始终显示文件的后缀名.

文件名后缀对操作系统的文件管理有一定的帮助, 但实际上操作系统在判断某个文件是否是某类文件时, 可能并不会单纯依赖文件后缀名, 还会根据文件的部分内容判断文件的类型. 同时, 对某些类型的文件而言, 后缀对文件的功能并没有太大影响. 比如C++的源文件, 实际上将其后缀改成其他的后缀或者直接删除, 也不会影响编译器的工作, 编译器仍然且只会根据文件的内容进行编译, 改变后缀只会改变代码的高亮(如果有)或者打开方式(默认打开方式可能将不能打开文件).

上面的例子也提示了, 修改后缀名并不会改变文件的格式, 后缀名仅仅作作为一种特殊的名字附在了文件最后, 并不会对文件内容有任何的影响, 因此修改后缀名以期改变文件格式是没有结果的. 如果需要改变文件格式, 请通过对应的方法进行. 这一点常见的误区便是修改图片的后缀(比如, 从`.png`到`.jpg`)之后便认为图片格式被改变了, 且图片依旧可以浏览. 实际上该方法并不会改变文件格式和内容, 能重新打开的原因完全是因为软件还识别了文件的一部分内容, 从而选择了合适的编码以打开文件.

这里还要提到的是Linux和Windows下文件后缀的区别. 一些后缀的文件, 如`.exe`, `.dll`, `.lib`等文件, 除了规定了文件的类型是二进制文件外, 还指明了该文件是适用于Windows平台的二进制文件. 这些文件在Linux平台下一般无法使用, 而Linux平台的二进制文件可能使用`.a`(静态链接库), `.so`(动态链接库)等后缀, 甚至可执行文件没有后缀. Linux常会根据文件开头的一些内容判断文件的类型. 但是也有 [论坛帖](#) 指出, Linux系统下的文件后缀在某些情况下仍是有其作用的.

最后, 请正确使用文件后缀, 并避免修改文件的后缀名, 以免造成不必要

的麻烦.

5.5 文件编码

上一节中介绍了文件后缀, 提到了“有些文件是单纯的文本文件(其实也并不单纯)”. 这里涉及到一些关于计算机如何将数据表示为人能够识别的信息, 如英文字母, 标点符号, 以及我们使用的汉字等等. 将计算机内部信息通过某种方式处理, 变成人类能够识别的内容或者能使用的内容的过程或是方式, 便是“编码”.

计算机对数据的编码方式决定了同一段数据最后呈现在人面前的形态, 最初的编码系统ASCII(American Standard Code for Information Interchange, 美国信息标准交换码)规定了计算机中数据与计算机操作, 字符, 数字等等之间的对应关系. 随后, 为了表示更多更复杂的内容, 各种不同的组织开发了自己的编码标准. 我国的推荐性国家标准 GB/T 2312-1980 中规定了一种中文的字符编码. 不同的标准虽然利好各自编码系统内部畅通交流, 但也引起了一些问题.

有一类非常常见的问题, 表现形式为从他人处收到的文件打开后发现是看不懂的乱码, 或者打开后通篇的“锱斤拷”. 这就是经典的编码错误问题, 因为打开或保存文件的人采用了错误的编码方式, 又被错误的打开后便会出现乱码. 除此之外, 不得不提的是微软的Windows记事本(Notepad)和文字编码之间的问题. Windows的记事本在过去会自动根据文件开头的内容判断文件的编码方式, 但该判断方式并不准确, 甚至常常造成问题, 导致文件的内容在输入时是一个样子, 在保存, 关闭, 打开后又是另一个样子. 这个问题可以见: [视频链接](#) 或者 [原视频链接](#). 这一问题在较新的Windows版本中已经得以修复.

20世纪80年代末, 美国加州出现的Unicode组织着手解决这一问题, 开发出UTF系列字符编码, 并随着长期的发展, 如今UTF-8编码成为互联网上的主要编码方式. 然而, 微软强大的本地化能力, 为日本地区操作系统带去 ¥ 作为文件路径分隔符的同时, 也为中国地区操作系统带来了默认使用的GB/T 2312-1980的编码标准. 为了代码的便于交流, 便于协作以及可维护性, 请将文件的编码设置为UTF-8格式编码. 同时, 当您打开一个文件发现内容全是乱码时, 请千万不要保存该文件, 转而尝试使用别的编码方式重打开该文件, 找

到正确编码方式再保存为新的正确的编码方式, 否则可能因为在错误编码下保存文件导致原数据被新编码的乱码字符所替代, 最终造成数据丢失. “锟斤拷”便是这样的错误编码保存循环中出现的.

5.6 控制字符

所谓“控制字符”(Control Sequence), 通常指用来控制计算机操作的字符. 观察键盘, 键盘上除了许多的“可打印字符”, 如数字, 字母, 符号等按键以外, 还有一些诸如 **Shift**, **Enter**, **↑**, **Home**, **Backspace** 等等具有功能的按键. 这些键按下后, 并不是直接让计算机进行对应的操作, 而是会将指令以字符形式传入, 并且交由计算机识别, 最后进行对应操作. 这类字符亦即所谓的“不可打印字符”. 比如在不支持`readline`库的C++控制台程序中, 输入的 **→** 并不会将光标向右移动, 反而会输入字符 `^[[C`, 而按下 **Delete** 也不会删除右方一个字符, 而是输入字符 `^[[3~`. 当尝试将这些字符打印在屏幕上时, 它们又不会被打印出来. 在C++中, 一部分的这类特殊字符可以用转义反斜杠 `\` 加一个字符的形式出现. 常见的比如换行符 `\n`, 回车符 `\r`, 制表符 `\t`, 字符串结尾符 `\0` 等.

有一些和操作系统相关的由特殊字符引起的问题, 其中最常见的是Mac OS, Linux与Windows三个操作系统对换行符的处理. 在Mac OS中换行符使用‘CR’(Carriage Return, `\r`), Linux 使用‘LF’(Line Feed, `\n`), 而Windows使用的是二者的组合, ‘CRLF’. 这种问题经常导致Windows下写成的或编辑保存过的Bash脚本不被Linux系统识别, 其常显示的错误为 “`.../bashM: bad interpreter...`”. 这便是由于Linux把CRLF下的CR认为是文本内容 `^M` 导致的. 因此, 请注意您的文件的行尾字符, 以防这类问题的发生.

5.7 面向对象

面向对象是一种编程思想, 是相对于面向函数、面向过程等概念而生的, 其基本思想是一切皆为“对象”, 对象内拥有必要的的数据以及可以操作对象的方法, 且可以自由创建, 销毁对象, 通过调用相关的方法操作对象以实现程序的目的. 相比于顺序式执行的面向过程编程思想, 或是“一切皆函数”的面向函数思想, 面向对象有着更加灵活, 层次分明的特点, 适合于开发大型程序.

在C++中面向对象通过类(class)实现.

5.8 C++语言标准

C++ 历经长时间发展,出现了许许多多不同的语言版本.不同的语言版本之间的主要差别为语言特性的支持以及一些库是否可用.目前最新的且较为广泛接受的语言版本为C++20,普遍应用的版本则为C++14, C++14标准同时也是Visual Studio 2022的默认语言标准,而常称的“现代C++”则一般指代C++11及以后的标准,因为C++11中引入了大量的更现代的语法特性.

MInDes 出于兼顾稳定性以及功能性的原因,选择使用C++17标准进行开发.

5.9 编译

编译(此处仅指C++的编译)是指将C++源代码通过编译器的处理生成一个可执行的程序的过程.编译过程随着程序规模,复杂程度而有时间上的变化,但其基本过程可以描述为以下几部分:

1. 将源文件编译为对象文件(.o)
2. 将对象文件按照源文件中所述的方式链接起来(此步骤需要链接器(linker))
3. 将链接后得到的中间文件再与外部的静态/动态链接库相连接(此步也需要链接器)
4. 生成二进制可执行文件

从上述过程可知,没有正确配置运行库环境可导致程序编译过程中产生链接器错误,例如找不到符号的定义等.因此开发环境的配置除了工具的设置外,配置正确的运行库也是环境配置的一部分.

5.10 编译器,MSVC, G++ 与 C++标准委员会

前面已经介绍了编译器的作用,也介绍了C++语言标准.语言标准由C++标准委员会商议确定,通过对提案的可行性审查,必要性审查等来确定标准C++在

某个标准下应该具有的语法特性. 该特性是推荐所有编译器共同支持的, 但因为各种原因, 部分语法特性并不是被所有编译器所支持的, 且每种不同的编译器也有各自不同的特点. 编译器可以实现非标准的C++语法特性, 为使用者提供便利, 但应注意使用了某款编译器特性的源代码, 在使用别的编译器时可能出现效率下降, 运行结果不符合预期, 甚至编译失败等各种问题. 其中, 处于Windows, Linux两大平台垄断地位的编译器, MSVC 与 G++, 均拥有大量的为适应各自平台而开发的非标准的语法特性. 在跨平台编程时, 应尽可能避免使用这类特性, 保持程序在双端都能有相似的表现.

MSVC编译器通常不直接接受编译器选项(flags / Compiler Options), 而是需要在VS中进行设置. 这类内容请查看Microsoft的各类文档, 如关于编译器选项的 [文档](#).

G++是GNU编译器套装 (GNU Compiler Collection, 简称GCC)中的编译器之一, 负责编译C++程序. 可以通过在命令行中添加编译器指令(flags)以控制编译器的行为, 也可以通过各类构筑系统(下文将提到)以控制编译器的行为.

MSVC编译器的语法与GCC的有很大区别, 但一般不会特意手动设置这些内容.

5.11 GNU Compiler Collection, GCC & G++

GNU编译器套装(GNU Compiler Collection)可以缩写为GCC, 而GCC本身也是个独立的程序, G++又被包含在GCC中, 这三者的关系较为复杂, 名称有其历史原因, 在此简要阐述以帮助读者辨别三者的区别.

GCC这个名称最早原意为GNU C Compiler, 即GNU C编译器, 是一款较为知名的C语言编译器, 相对应的还有C++对应的G++, 即GNU C++编译器. 而随着GNU社区的不断壮大, GNU团队开发的编译器开始不仅限于C语言或者C++等几个语言, 如 ObjectiveC, Fortran, Ada等等的编译器. 于是, GNU将他们开发出的这些语言的编译器以及这些编译器依赖的库(libstdc++等)合并起来, 并称为GNU Compiler Collection, 并重新将GCC的含义变为该义.

GCC即便含义发生了转变(由编译器到编译器集合), 作为可执行程序的编译器工具链而言, GCC仍然是可用的, 但在编译C++程序时应该使用G++而非GCC. 详细请参阅 [Using the GNU Compiler Collection](#).

总而言之, 目前提到的GCC应被理解为GNU编译器套装, 可以直接用以编译C语言, 可以兼职编译C++. G++作为GNU编译器套装中的一部分, 是专职编译C++语言的编译器. 在编译C++程序时应优先尝试用G++进行编译.

5.12 代码构筑系统, CMake

一般的小型程序可以通过编译器简单的处理即可编译为可执行的二进制程序, 但当代码更加复杂, 文件众多且相互关联之时, 有一个辅助进行编译的程序可以有效地提高程序开发效率. 这类辅助程序即代码构筑系统. 目前常见的构筑系统有Microsoft VS所采用的MSBuild构筑系统, Linux系统自带的Makefile, 广受欢迎的CMake, 图形化界面Qt所采用的QMake等等. 代码构筑系统为管理大型程序的编译, 调试等提供了便利. 目前MInDes采用MSBuild(Windows)与CMake(Linux)两种方式进行构建.

CMake的使用已经高度集成在VSCode中, 通过CMake插件提供的GUI可以简单实现CMake的各种操作. 然而, 依旧可以使用命令行进行操作. CMake的使用离不开 *CMakeLists.txt* 的编写, 通过插件可以实现基础的文件写入, 但为了实现更复杂的功能, 仍然需要自行编辑*CMakeLists.txt*. 此处不再详细介绍 CMake 以及 *CMakeLists.txt* 的使用, 感兴趣可以查看CMake的文档, 但因为CMake为兼容并包多种写法, 文档也较为混乱, 建议必要时使用ChatGPT等AI工具辅助生成*CMakeLists.txt*.

5.13 代码调试(Debug)

代码一次写成, 运行良好固然很好, 但这种情况在实际开发中很难遇到. 实际开发中常会遇到各种各样的问题阻碍开发进展. 这些程序中或逻辑或语法的错误就被称为Bug, 在程序中排查Bug并修正以使程序得以正常运行的过程即是调试, 亦即所谓的Debug.

Debug这一过程所占用的开发时间极长, 据统计几乎占据了实际开发用时的50%以上, 因此拥有一定的Debug技巧将极大提高工作效率, 某种程度上也可以降低开发者的心智负担. 最简单的调试方法即将程序在某一步的数据通过 `cout` 或 `printf` 输出到控制台上. 然而该方法的问题在于每次调试都需要改变代码, 重新编译, 然后再等待程序运行至对应的位置再将内容输出.

该方法具有多种局限性, 并不具备足够的实用性.

更现代化的技巧则为使用调试器(Debugger)运行程序来实现类似于Python逐行解释运行的效果. VS中自带的MSVC已有调试器集成在内部, 而Linux平台下最常用到的Debugger即为gdb(GNU Debugger). 在调试器的辅助下, 可以在源代码中添加“断点”(一般表现为代码左端的红色圆点), 当程序在调试模式下编译后交由调试器启动时(通常IDE或CMake会将该步骤简化为一步), 程序运行至断点所在行时便会停下, 不执行该行转而等待用户的命令. 此时可以通过调试控制台(Debug Console)和Debugger交互, 或者查看程序中变量的值, 甚至修改变量的值. 同时可以查看函数的调用栈情况, 也可以使用“步进”(Step Over)逐行运行, 或者“步入”(Step Into)查看该行函数内部的运行情况, 亦或者“步出”(Step Out)跳出该函数回到上一层函数调用栈. Debugger的运用使程序的调试过程变得更加便利.

5.14 并行运算, OpenMP与MPI

并行运算是指通过将程序的运算工作按照某种方式拆分后分配给不同的算力单元进行运算, 以此充分提高运算效率, 减少资源浪费. 目前常见的并行运算方式有 OpenMP(Open Multi Process)这样多线程共享内存的并行方式, 也有MPI(Message Passing Interface)这样的多进程共享内存的并行方式.

进程指代一个正在运行中的程序, 是系统资源调度分配的基本单位, 每个进程拥有自己的内存空间(即自己拥有独立的空间用以存储数据), 而线程是进程的子任务, 是CPU调度分配资源的基本单位, 其并不单独占有某个内存空间, 而是从进程中获取自己所需的内存资源. 通过共享内存, 多个线程或进程之间即可交换数据, 比如将一个大的数据划分为若干份, 每一份数据由一个线程(OpenMP)或一个进程(MPI)持有, 各个线程或进程之间有一定的数据交换以保证数据的完整性. 在经过每个线程或进程对数据处理后, 再将处理好的结果汇总, 按照初始划分方式重新拼合起来. 线程所共享的内存空间只能是某一整个完整内存空间(所在的进程的内存空间), 并不能跨内存空间交换数据, 因而OpenMP仅能支持单个内存空间内进行并行运算. 而MPI由于设计之初即为在多个进程之间交换数据, 亦即在多个独立的内存空间之间交换数据, 绕过了线程不能跨内存空间的局限性. 目前大部分超算都支持MPI并行

的程序.

OpenMP相比于MPI, 速度和可运算规模上MPI更加强大, 但MPI为实现多进程, 程序内部逻辑经常晦涩难懂, 且对程序的调试相较于OpenMP程序而言会复杂很多, 需要设置一个主进程后令Debugger跟踪该主进程, 极大提高了调试的门槛, 且并不方便, 而OpenMP只需要在编译过程中给予编译器相关的设置, 便可以直接开始并行运算而无需复杂的操作. 出于这样的考虑, 目前MInDes使用了OpenMP进行并行运算. 但未来可能考虑使用MPI以进一步提高软件的运算效率.

5.15 Linux 内核与Linux 系统

严格来讲, Linux应该指一种由Linus Torvalds所开发的操作系统内核, 意在模仿当时由贝尔实验室开发的Unix操作系统, 实现一个完全自由的类Unix系统. 如今也被用以指代使用Linux内核的操作系统大类. 由于Linux内核仅作为操作系统的内核, 不同的组织或个人开发了不同的基于该内核的不同操作系统, 即为所称的Linux发行版. 目前有众多特性各异, 适用范围不同的发行版, 常见的个人发行版为Ubuntu系统, 该发行版也是MInDes的主要测试目标系统.

5.16 操作系统, WSL & 虚拟机

本程序的开发主力环境为Windows平台, 目前在Windows10与Windows11平台上进行开发. 为使程序可以运行在算力更高的服务器或超算上, 本程序兼容了主流的Linux平台. 目前测试过的Linux发行版有Ubuntu与Arch Linux. 为了在Windows机器上可以测试Linux环境下程序运行情况, 一般的做法为使用虚拟机或者WSL (Windows Subsystem Linux) 模拟出Linux操作系统环境.

其中, 虚拟机是一种成熟的技术, 通过在宿主机上的虚拟机软件内安装一个完整的Linux发行版来获得Linux环境. 而WSL则是微软所开发的一种Linux子系统, 和虚拟机相比, WSL的独立性不足, 但大部分文件是可以简单地共享的. 对本程序而言, WSL足以胜任大部分的Linux环境测试工作, 因而推荐使用WSL.

5.17 命令行, 终端, Shell, Powershell & Bash

开发程序的过程中, 尤其是在操作Linux系统时, 经常绕不开命令行操作. 命令行界面(Command Line Interface, CLI)是一种操作电脑的界面, 通过输入文字命令以操作电脑执行命令, 通常程序初学者运行程序后出现的“黑框”即作为一种CLI. 与之相对的概念即为图形用户界面(Graphical User Interface, GUI). MInDes程序的操作建立在CLI上.

CLI需要有载体软件呈现, 这类载体被称为终端(Terminal). 最早的终端是一类实体机器, 通过终端输入命令来与计算机交互. 如今终端已经通过终端模拟器内置于电脑中, 成为了一个软件. 与终端相关的概念为“控制台”(Console), 历史上二者有一定的区别, 现在大多不再区分.

Shell是为用户提供操作电脑方法的程序, 在计算机层次架构中处于用户交互与底层运算之间的位置. 在不同的平台上均有众多各自不同的Shell程序, Windows端如传统的cmd.exe, 较为现代化的Windows Powershell, 以及微软最新推出的Powershell-7, 而Linux端则有常见的随系统安装自带的Bash, 更加现代化, 自定义程度更高, 与Bash相兼容的zsh, 拥有与Bash脚本不同语法, 更加人性化的fish等等.

Shell与命令行界面、终端之间的关系是: 终端内可运行多种Shell程序, Shell程序呈现出的界面为命令行界面. 在一般交流中三者区别并不明显, 影响交互方式的主要为Shell程序, 终端主要作为不同Shell程序的载体. 在Windows端建议使用微软最新推出的Windows Terminal, 并且在Shell程序中推荐命令与Linux有相似之处的Powershell-7. 而在Linux端, 已有的Bash已经具有较为强大的功能, 足以应对绝大部分场景的使用.

6 Q&A

此处回答一些可能会有的问题, 同时也是作为前面内容的补充.

6.1 我是搞材料的, 学这么多计算机的软件的内容做什么?

本手册面向的读者群体为希望加入MInDes程序开发, 但软件开发相关领域了解不深的科研工作者. 为尽可能减轻读者负担, 笔者尽己所能将内容压

缩至精简且易于理解的程度. 这些内容或问题在软件开发的过程中会频繁遇到, 因此具有一定的了解会让开发工作更加顺利.

笔者希望读者能在该手册中快速检索到自己需要的信息, 不奢求逐行看完.

6.2 你写的一大堆全都是泛泛而谈, 有没有更专业的文档?

这里提供一份清单, 内容是笔者认为较好的软件官网, 教程指南, 以及别的优秀资源. 前述的链接也被汇总至此.

首先是软件官网下载与安装教程:

- [Microsoft VS 官网](#) : 可供下载Microsoft Visual Studio.
- [Microsoft VSCode 官网](#) : 可供下载Microsoft Visual Studio Code.
- [WSL 官方安装指南](#) : 可按照微软给出的安装教程安装WSL.
- [WSL 官方开发环境设置指南](#) : 可按照微软给出的安装教程设置WSL开发环境.
- [Paraview 官网](#) : 可供下载Paraview.
- [Git 下载页面](#) : 可下载供Windows平台下运行的Git客户端.
- [Windows Terminal 官方下载指南](#) : 可按照微软给出的教程安装Windows Terminal, 实现在一个窗口打开多个Shell.
- [Powershell-7 官方下载指南](#) : 可按照微软给出的教程安装Powershell-7.
- [MInDes-LSP](#) : 笔者的VSCode插件: MInDes-LSP的GitHub仓库, 可以为MInDes的输入文件提供语法高亮等功能.

以下是一些文档和优秀的操作教程, 有官方出品的, 也有个人写作的:

- [VS 文档集](#) : 可查阅Visual Studio的方方面面.
- [VSCode 文档集](#) : 可查阅Visual Studio Code的各项使用方法.

- [Git 文档集](#) : Git 的官方文档.
- [Pro Git 电子书](#) : Git官方出品的Git教程电子书, 涵盖了Git的方方面面.
- [廖雪峰的Git教程](#) : 简单鲜活的Git教程, 主要以命令行模式使用Git.
- [cppreference](#) : 非常好用的C++ API 参考, 记录有大量的C++在不同语言标准下的语言API, 可信度很高.
- [cplusplus](#) : 同上, 优秀的C++ API 参考, 同时拥有一些教程和文章以供学习.
- [ArchWiki](#) : Arch Linux团队维护的百科, 除了介绍了Arch Linux外也介绍了许多Linux系统甚至内核的相关知识, 可在需要时查阅.
- [Linux man pages](#) : Linux系统中用到的命令, 函数等的用户手册线上全集. 除了该线上版本外, Linux系统中通过命令 `man` 也可以调出该文档的离线版本.
- [GNU Manuals Online](#) : GNU团队出品的开源自由文档, 记录了GNU Project中软件的用户手册, 如Bash, GCC等.
- [ISO C++ 官网](#) : C++标准官网. 内含许多有助于开发的信息.
- [OpenMP 学习资源](#) : OpenMP官网提供的OpenMP相关教程.

以下是一些优秀的工具类网站, 以及优秀的论坛, 在开发过程中可能可以用到:

- [Quick Benchmark](#) : 一个可以对代码运行速度进行检测的网站, 可以用以对比两段代码的运行速度差距以便优化算法.
- [Compiler Explorer](#) : 一个强大的C++编译器整合网站, 可以从网站中查到不同编译器之间对代码的处理区别, 提供了代码的汇编结果.
- [RegExr](#) : 一个可以测试正则表达式结果的网站, 对Linux的部分使用场景或是需要用到正则表达式的场景会有帮助.

- [Wikipedia](#) : 维基百科, 可以简单轻松获得某个词条的基础信息.
- [ChatGPT](#) : ChatGPT, 强大的AI搜索问答工具.
- [Stack Overflow](#) : 著名程序员问答论坛, 近乎任何程序问题都可以从此处寻得专业且切实有效的答案.
- [Stack Exchange](#) : 上述论坛的母站. 热门的论坛除了Stack Overflow外, 还有Mathematics, Super User, Ask Ubuntu等等, 帖子质量都很高.
- [GitHub](#) : 知名代码托管平台, 拥有海量开源项目.

此外, 是一些个人认为有趣的资源或内容:

- [Git的故事](#) : 讲述了Git的前世今生, 以及GitHub的诞生.
- [Linux中国](#) : 一个已经停运的Linux开源社区, 有很多很好的技术贴子, 尤其是Linux相关的帖子.
- [阮一峰的网络日志](#) : 著名科技圈周刊, 每周五更新, 支持RSS订阅.
- [PowerToys官网](#) : Microsoft官方出品的Windows平台强大的工具合集.
- [Everything官网](#) : 一款强大的Windows端文件搜索工具, 与Windows自带的搜索功能相比搜索内容更快更全.
- [菜鸟教程](#) : 一个编程学习教程网站. 与上面的教程相比略显浅薄, 但也足以入门.
- [廖雪峰的主页](#) : 廖雪峰的个人主页, 拥有许多优秀的教程, 如Python教程, Java教程等. 上述的Git教程是其主页内容的一小部分.

6.3 你写的这些长到看不下去, 有没有省流版?

如果您具有优秀的互联网搜索能力, 且对章节5基础概念部分的大多数概念都具有或浅或深的了解, 使用本手册目的仅为快速同步设置好开发环境, 下面是一份您需要注意的内容:

- 在Windows上下载并安装VS, VSCode, GitHub Desktop (可选地, 安装Powershell-7和Windows Terminal, 安装Git替代GitHub Desktop). 安装VS时注意选择安装 C++桌面开发(Desktop Development with C++)的套件.
- 使用GitHub Desktop或Git, 将MInDes或MInDes_DEV的GitHub仓库下载到本地.
- 创建一个VS项目, 将下载到本地的MInDes仓库内的文件添加至这个项目中. 请注意, 只添加程序的源文件. 随后为该项目设置C++标准到C++17, 并打开OpenMP的支持.
- 将本地仓库中的如下三个文件:*libfftw3-3.dll*, *libfftw3f-3.dll*, *libfftw3l-3.dll* 复制到项目文件夹下以便链接器寻找到动态链接库 (可选地, 自行设置动态链接库的寻址). 至此, Windows端的VS配置就完成了.
- 根据官方教程安装WSL, 然后在VSCode中安装对应插件以远程连接至WSL. 在VSCode中安装C++与CMake的插件 (可选地, 安装MInDes-LSP).
- 在WSL中安装G++, GDB, CMake, *readline*, 用VSCode打开MInDes或MInDes_DEV的本地仓库, 用CMake尝试运行程序. (可选地, 安装fftw3开发用库到本地并修改*CMakeLists.txt*内容以改变fftw库的寻址).

完成上述步骤, 您应该可以在一台新组装的电脑上搭建开发MInDes的必要环境. 请注意, 上述步骤中每一步均可能遇到程度不等的问题, 请善用搜索引擎. 遇到的任何问题也欢迎提出.

6.4 感觉看完还是云里雾里的...

不得不承认的一点是, 作为实操手册而言, 编写本手册最佳的方式应为一边操作一边记录具体的操作, 以力求读者对照本手册操作时具有足够的可行性. 但笔者的环境已经配好, 推倒重来并不容易, 因此希望读者在使用本手册时能够对照每一步操作, 如有不妥当的步骤请联系我们.

6.5 这么多页,一幅图都没有?

由于本文采用 $\text{\LaTeX}$ 写成,其图片的处理十分麻烦,而笔者能力着实有限,因此您看到的早期版本可能是没有或仅含有少量图片的.在更后期的版本中可能会加入更多的图片说明,也可能考虑采用MarkDown重写本手册以便于更新或加入图片.

6.6 我觉得你写得不好,我要自己写/修改/增补内容.

欢迎!您可以联系笔者以免费获得一份最新版的本手册的 $\text{\LaTeX}$ 代码.该文档如开发到足够成熟,可能会考虑将之开源,届时会放在某个GitHub仓库中.本手册将持续更新,还请各位使用者不吝赐教.

6.7 我看了这个手册,也搞定了里面讲的配置,但是还是不知道怎么开始做MInDes开发,这个不是开发入门手册吗?

这个手册的初衷其实是为了帮助读者搭建好MInDes开发的环境配置,但因涉及内容越来越多,才更名为开发入门手册.关于实际开发过程的讲解可能会安排培训,更新本手册或者出新的,专注于开发指导的手册.